UNIVERSITÉ — DÉPARTEMENT INFORMATIQUE

Projet BigData

Infrastructure Hadoop, MapReduce & Analyse PySpark

Composantes du projet

Partie 1 : Cluster Hadoop
(HDFS + MapReduce via Docker)
Partie 2 : Analyse de données
salariales avec Apache Spark

Visualisation : Dashboard Streamlit interactif

Auteur : Jasser Houimli – 2BD1

Année : 2025–2026

Module : Big Data & Technologies Distribuées

Docker · Hadoop 3.2.1 · Apache Spark 3.5.1 · PySpark · Streamlit

Résumé

Résumé. Ce rapport présente la mise en place d'une infrastructure Hadoop via Docker, ainsi qu'une analyse de données à l'aide d'Apache Spark. Le document couvre l'installation, le déploiement, des exemples d'utilisation, et des résultats analytiques accompagnés d'un tableau de bord interactif.

Table des matières

1 Introduction Générale **3**

2 Partie 1 - Infrastructure Hadoop avec Docker **4**
2.1 Installation de l'environnement 4
2.1.1 Installation des dépendances 4
2.1.2 Vérification des versions installées 5
2.2 Architecture du Cluster Hadoop 5
2.2.1 Fichier docker-compose.yml 5
2.2.2 Démarrage du cluster 8
2.2.3 Cluster en fonctionnement - Interface HDFS 8
2.3 Traitement MapReduce - Comptage des achats par ville 9
2.3.1 Le dataset : purchases.txt 9
2.3.2 Copie des données vers HDFS 9
2.3.3 Implémentation du Mapper 10
2.3.4 Implémentation du Reducer 10
2.3.5 Exécution du job Hadoop Streaming 11
2.3.6 Résultats du job MapReduce 12

3 Partie 2 - Analyse de Données avec Apache Spark **13**
3.1 Ajout de Spark au cluster Docker 13
3.1.1 Extension du docker-compose.yml 13
3.1.2 Démarrage des conteneurs Spark 14
3.1.3 Interface Web Spark Master 14
3.1.4 Installation des bibliothèques Python 15
3.2 Description du Dataset — Job Salary Prediction 15
3.2.1 Source Kaggle 16
3.3 Script d'Analyse PySpark (spark.py) 16
3.3.1 Initialisation de la SparkSession 17
3.3.2 Lecture et inspection du CSV 18
3.3.3 Aperçu des données brutes 19
3.4 Analyse Descriptive — KPIs Globaux 19

3.5 Résultats d'Analyse Détaillés 20
3.5.1 Top 10 des postes par salaire moyen 20
3.5.2 Top 10 des localisations par salaire moyen 21
3.5.3 Salaire moyen selon le statut de télétravail 22
3.5.4 Salaire moyen selon le niveau d'éducation 23
3.5.5 Top 10 des secteurs d'activité par salaire moyen 24
3.5.6 Salaire moyen par nombre de certifications 25
3.5.7 Tableau de bord Streamlit 26

4 Analyse Prescriptive **28**
4.1 Pour les Entreprises 28
4.2 Pour les Étudiants 28
4.3 Pour les Équipes RH 28

5 Conclusion **29**
5.1 Bilan du projet 29
5.2 Principaux enseignements 29

Références et Bibliographie **30**

Introduction Générale

> Ce rapport présente un projet complet de **Big Data** réalisé en deux parties complémentaires : une infrastructure distribuée Hadoop déployée via Docker, et une analyse approfondie de données salariales à l'aide d'Apache Spark, complétée par un tableau de bord interactif Streamlit.

Le Big Data désigne l'ensemble des technologies permettant de stocker, traiter et analyser des volumes de données massifs. Les principaux défis sont décrits par les **5 V** : Volume, Vitesse, Variété, Véracité et Valeur. Dans ce projet nous mettons en œuvre les deux principales technologies de l'écosystème Hadoop : HDFS, MapReduce et Apache Spark.

Partie 1 — Infrastructure Hadoop avec Docker

2.1 Installation de l'environnement

2.1.1 Installation des dépendances

La première étape consiste à installer tous les outils nécessaires : Docker, Docker Compose, Python 3, pip et Java (OpenJDK 11) sur la machine hôte (Ubuntu 24).

```
sudo apt install docker.io docker-compose python3 python3-pip \
                 python3-venv openjdk-11-jdk wget
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker $USER
```

Listing 2.1 – Installation des dépendances système

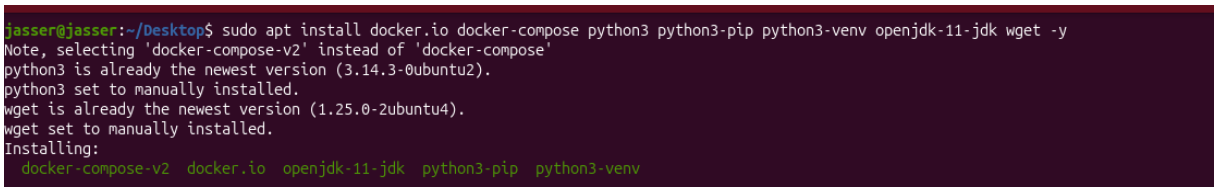


FIGURE 2.1 – Installation de Docker, Python, Java via apt

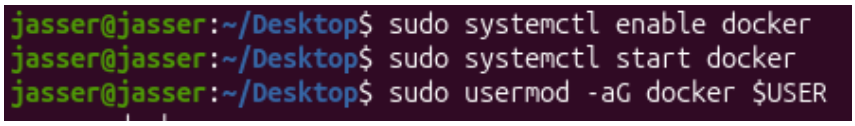


FIGURE 2.2 – Activation et démarrage du service Docker

2.1.2 Vérification des versions installées

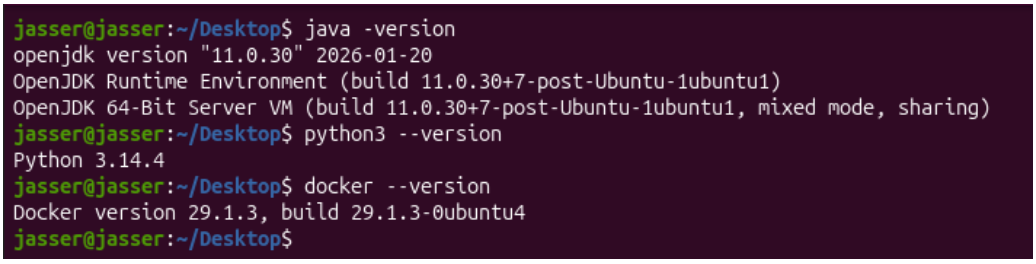


FIGURE 2.3 – Versions vérifiées : Java 11.0.30, Python 3.14.4, Docker 29.1.3

Les versions confirmées sont :

— **OpenJDK** 11.0.30 — 2026-01-20

— **Python** 3.14.4

— **Docker** version 29.1.3, build 29.1.3-0ubuntu4

2.2 Architecture du Cluster Hadoop

2.2.1 Fichier docker-compose.yml

Le cluster Hadoop est entièrement conteneurisé avec Docker Compose. Il comprend 1 namenode, 3 datanodes, un resourcemanager, un nodemanager, un secondarynamenode et un historyserver.

root@Jasser: ~/bigdata

```
  GNU nano 6.2
version: "3"

services:

  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    hostname: namenode
    ports:
      - "9870:9870"
      - "9000:9000"
    environment:
      CLUSTER_NAME: projet
      CORE_CONF_fs_defaultFS: hdfs://namenode:9000
      YARN_CONF_yarn_resourcemanager_hostname: resourcemanager
    networks:
      - hadoop

  datanode1:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode1
    hostname: datanode1
    environment:
      CLUSTER_NAME: projet
      CORE_CONF_fs_defaultFS: hdfs://namenode:9000
    depends_on:
      - namenode
    networks:
      - hadoop

  datanode2:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode2
    hostname: datanode2
    environment:
      CLUSTER_NAME: projet
      CORE_CONF_fs_defaultFS: hdfs://namenode:9000
    depends_on:
      - namenode
    networks:
      - hadoop

  datanode3:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode3
    hostname: datanode3
    environment:
      CLUSTER_NAME: projet
      CORE_CONF_fs_defaultFS: hdfs://namenode:9000
    depends_on:
      - namenode
    networks:
      - hadoop

  resourcemanager:
    image: bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8
    container_name: resourcemanager
    hostname: resourcemanager
    ports:
      - "8088:8088"
```

FIGURE 2.4 – Code du fichier `docker-compose.yml` — services namenode, datanodes, resource-

```yaml
version: "3"
services:
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    hostname: namenode
    ports:
      - "9870:9870"
      - "9000:9000"
    environment:
      CLUSTER_NAME: projet
      CORE_CONF_fs_defaultFS: hdfs://namenode:9000
      YARN_CONF_yarn_resourcemanager_hostname: resourcemanager
    networks:
      - hadoop

  datanode1:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode1
    hostname: datanode1
    environment:
      CLUSTER_NAME: projet
      CORE_CONF_fs_defaultFS: hdfs://namenode:9000
    depends_on:
      - namenode
    networks:
      - hadoop

  # datanode2 et datanode3 : m me configuration

  resourcemanager:
    image: bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8
    container_name: resourcemanager
    hostname: resourcemanager
    ports:
      - "8088:8088"
    environment:
      CLUSTER_NAME: projet
      CORE_CONF_fs_defaultFS: hdfs://namenode:9000
    depends_on:
      - namenode
    networks:
      - hadoop

networks:
  hadoop:
    driver: bridge
```

Listing 2.2 – docker-compose.yml — extrait namenode et datanodes

2.2.2 Démarrage du cluster

```
cd ~/bigdata
docker compose up -d
```

Listing 2.3 – Lancement du cluster



FIGURE 2.5 – Lancement de `docker compose up -d` — pull des images



FIGURE 2.6 – Les 11 conteneurs démarrés avec succès (`Started`)

2.2.3 Cluster en fonctionnement — Interface HDFS

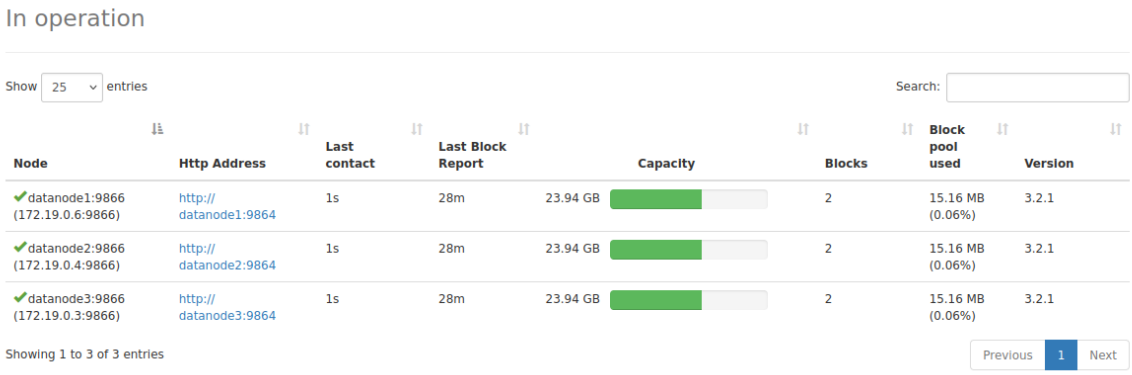


FIGURE 2.7 – Interface web HDFS — les 3 datanodes actifs avec 23.94 GB de capacité chacun

Les 3 datanodes sont visibles dans l'interface HDFS (`http://localhost:9870`) :

— **datanode1** — 172.19.0.6 :9866 — Capacité : 23.94 GB

— **datanode2** — 172.19.0.4 :9866 — Capacité : 23.94 GB

— **datanode3** — 172.19.0.3 :9866 — Capacité : 23.94 GB

2.3 Traitement MapReduce — Comptage des achats par ville

2.3.1 Le dataset : purchases.txt

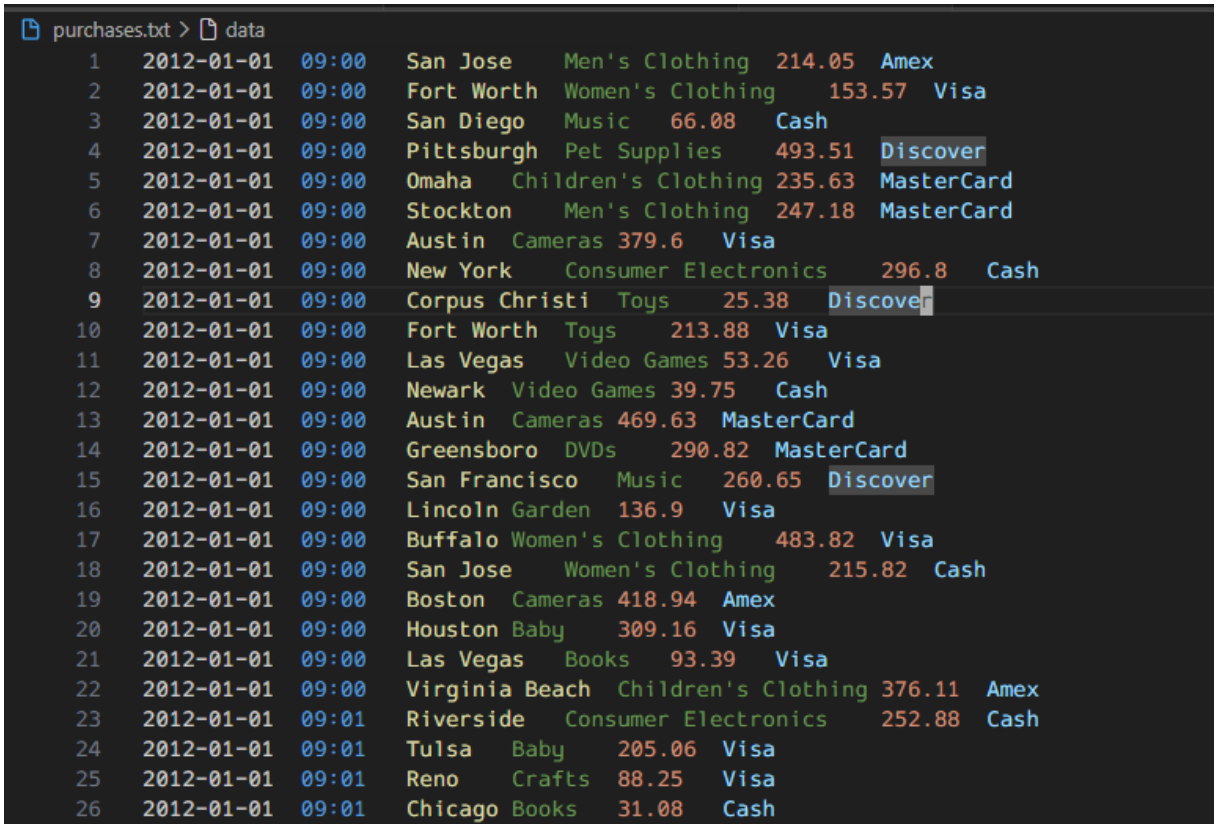


FIGURE 2.8 – Aperçu du fichier `purchases.txt` — transactions avec date, heure, ville, catégorie

Le fichier `purchases.txt` contient des milliers de transactions commerciales (15.7 MB). Chaque ligne est tabulée : `date \t heure \t ville \t catégorie \t montant \t paiement`.

2.3.2 Copie des données vers HDFS

```
docker cp purchases.txt namenode:/tmp/
docker exec -it namenode hdfs dfs -mkdir -p /input
docker exec -it namenode hdfs dfs -put -f /tmp/purchases.txt /input/
docker exec -it namenode hdfs dfs -ls /input
```

Listing 2.4 – Copie de purchases.txt dans HDFS

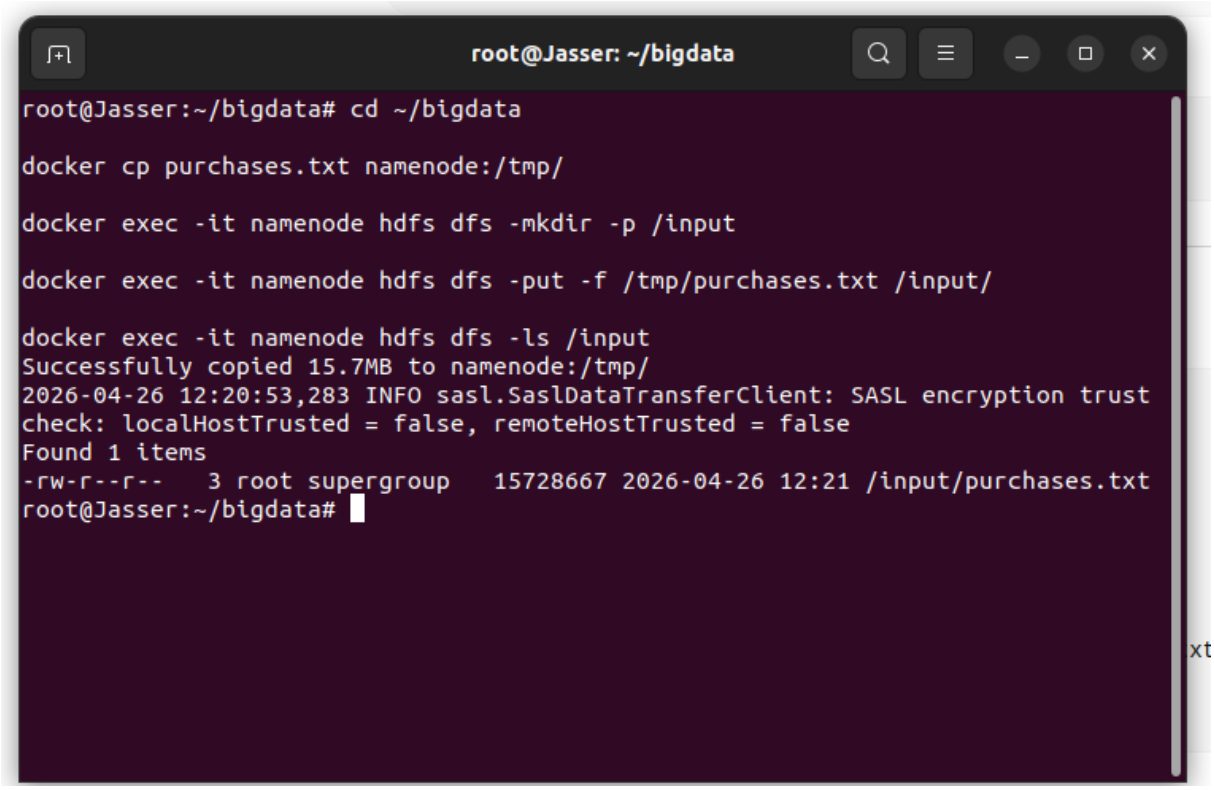


FIGURE 2.9 – Copie de `purchases.txt` (15.7 MB) vers le namenode puis vers HDFS

2.3.3 Implémentation du Mapper

Le Mapper lit chaque ligne, extrait la ville (colonne index 2) et émet `(ville, 1)` :

```python
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()
    parts = line.split("\t")

    if len(parts) >= 3:
        city = parts[2].strip()
        print("{}\t1".format(city))
```

Listing 2.5 – mapper.py

2.3.4 Implémentation du Reducer

Le Reducer reçoit les paires triées par ville et additionne les compteurs :

```python
#!/usr/bin/env python3
import sys

current_key = None
current_sum = 0

```

```
for line in sys.stdin:
    line = line.strip()
    key, value = line.split("\t")
    value = int(value)

    if current_key == key:
        current_sum += value
    else:
        if current_key:
            print("{}\t{}".format(current_key, current_sum))
        current_key = key
        current_sum = value

if current_key:
    print("{}\t{}".format(current_key, current_sum))
```

Listing 2.6 – reducer.py

2.3.5 Exécution du job Hadoop Streaming

```
docker cp mapper.py  namenode:/tmp/mapper.py
docker cp reducer.py namenode:/tmp/reducer.py
docker exec -it namenode bash

STREAMING_JAR=$HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming
   -*.jar
hadoop jar $STREAMING_JAR \
    -input  /input/purchases.txt \
    -output /output_city \
    -mapper  "/tmp/mapper.py" \
    -reducer "/tmp/reducer.py"
```

Listing 2.7 – Exécution MapReduce via hadoop-streaming

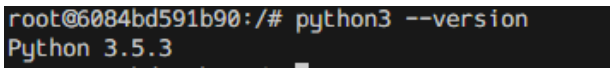


FIGURE 2.10 – Vérification de Python 3.5.3 installé dans le conteneur namenode

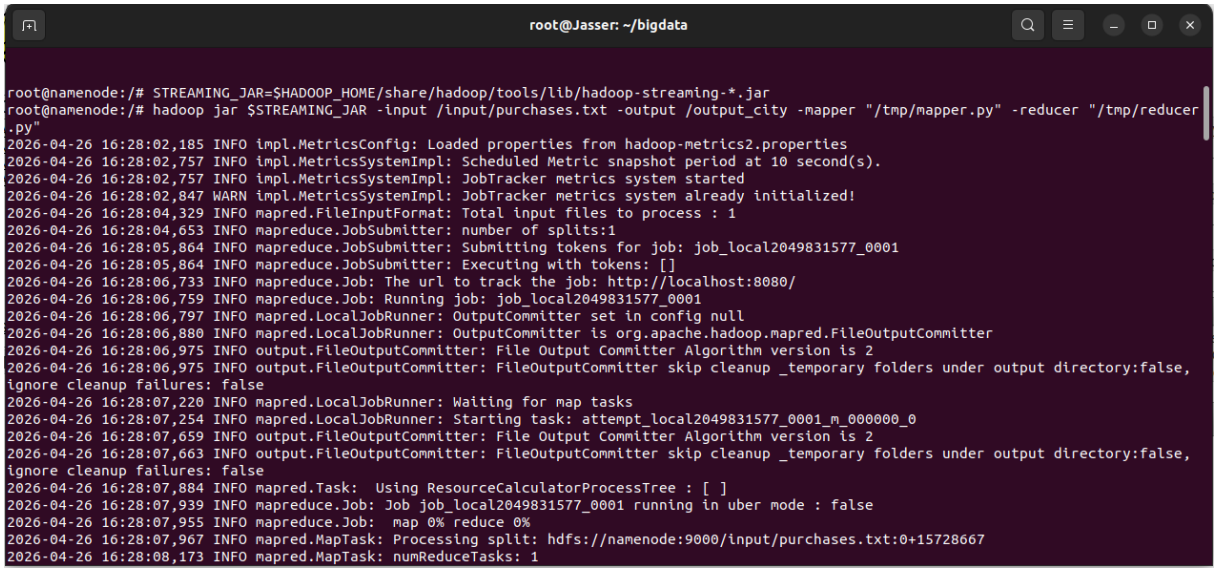


FIGURE 2.11 – Exécution du job Hadoop Streaming — logs d'avancement du job MapReduce

2.3.6 Résultats du job MapReduce

```
hdfs dfs -ls /output_city
hdfs dfs -cat /output_city/part-00000
```

Listing 2.8 – Affichage des résultats

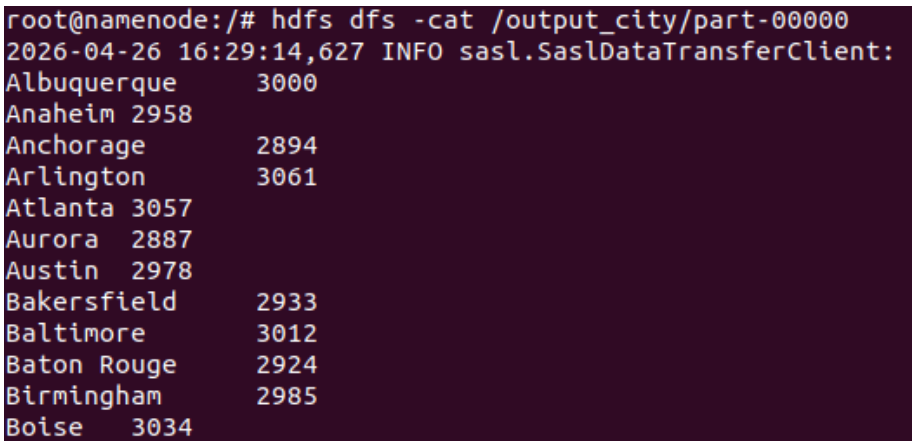


FIGURE 2.12 – Résultats MapReduce — nombre de transactions par ville (extrait alphabétique)

Extrait des résultats obtenus : Albuquerque (3000), Arlington (3061), Atlanta (3057), Baltimore (3012), Boise (3034)…

Partie 2 — Analyse de Données avec Apache Spark

3.1 Ajout de Spark au cluster Docker

3.1.1 Extension du docker-compose.yml

```
  spark-master:
    image: apache/spark:3.5.1
    container_name: spark-master
    hostname: spark-master
    user: "0:0"
    volumes:
      - ./scripts:/opt/scripts
      - ./datasets:/opt/data
    command:
      - /opt/spark/bin/spark-class
      - org.apache.spark.deploy.master.Master
      - --host
      - spark-master
      - --port
      - "7077"
      - --webui-port
      - "8080"
    ports:
      - "7077:7077"
      - "8080:8080"
    depends_on:
      - namenode
    networks:
      - hadoop

  spark-worker-1:
    image: apache/spark:3.5.1
    container_name: spark-worker-1
    user: "0:0"
    command:
      - /opt/spark/bin/spark-class
      - org.apache.spark.deploy.worker.Worker
```

```
      - spark://spark-master:7077
    depends_on:
      - spark-master
    networks:
      - hadoop

  # spark-worker-2 et spark-worker-3 : m me configuration
```

Listing 3.1 – Services Spark ajoutés au docker-compose.yml

3.1.2 Démarrage des conteneurs Spark

```
✔ Container datanode2          Running                                    0.0s
✔ Container datanode1          Running                                    0.0s
✔ Container spark-master       Started                                   18.8s
✔ Container datanode3          Running                                    0.0s
✔ Container resourcemanager    Running                                    0.0s
✔ Container spark-worker-2     Started                                   17.8s
✔ Container spark-worker-1     Started                                   17.9s
✔ Container spark-worker-3     Started                                   18.0s
✔ Container nodemanager1       Started                                    1.7s
root@Jasser:~/bigdata# docker logs spark-master
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/04/29 11:34:28 INFO Master: Started daemon with process name: 1@spark-master
26/04/29 11:34:28 INFO SignalUtils: Registering signal handler for TERM
26/04/29 11:34:28 INFO SignalUtils: Registering signal handler for HUP
26/04/29 11:34:28 INFO SignalUtils: Registering signal handler for INT
26/04/29 11:34:35 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java c
lasses where applicable
26/04/29 11:34:38 INFO SecurityManager: Changing view acls to: root
26/04/29 11:34:38 INFO SecurityManager: Changing modify acls to: root
26/04/29 11:34:38 INFO SecurityManager: Changing view acls groups to:
26/04/29 11:34:38 INFO SecurityManager: Changing modify acls groups to:
26/04/29 11:34:38 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view per
missions: root; groups with view permissions: EMPTY; users with modify permissions: root; groups with modify permission
s: EMPTY
26/04/29 11:34:40 INFO Utils: Successfully started service 'sparkMaster' on port 7077.
26/04/29 11:34:41 INFO Master: Starting Spark master at spark://spark-master:7077
```

FIGURE 3.1 – Démarrage des conteneurs Spark — spark-master, spark-worker-1/2/3 `Started`

3.1.3 Interface Web Spark Master

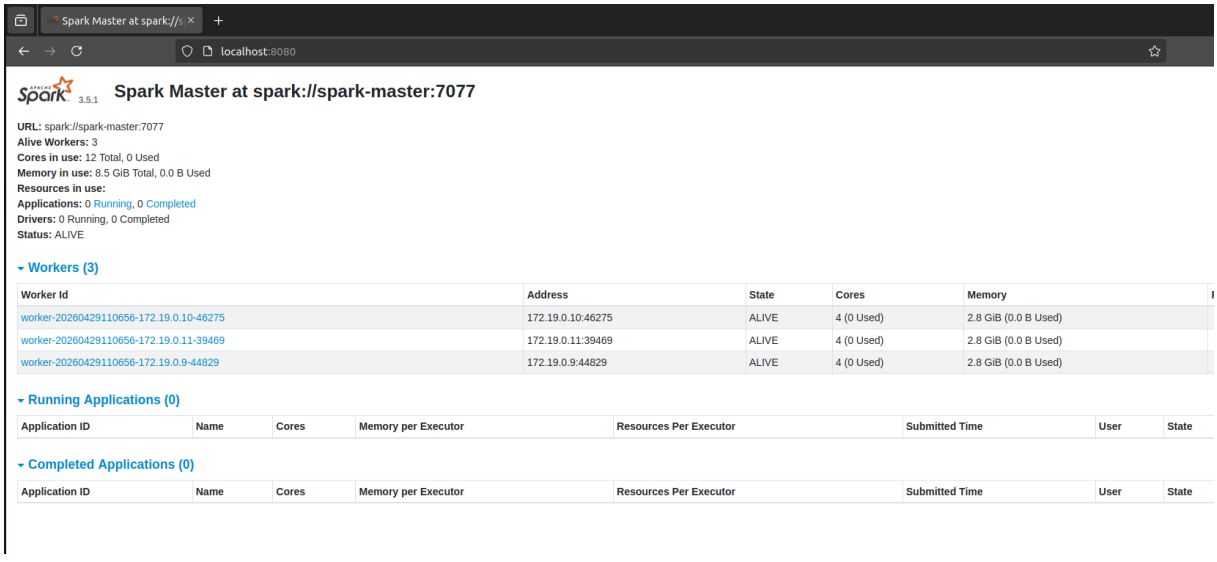


FIGURE 3.2 – Spark Master UI (`http://localhost:8080`) — 3 workers ALIVE, 12 cœurs, 8.5 GiB

3.1.4 Installation des bibliothèques Python

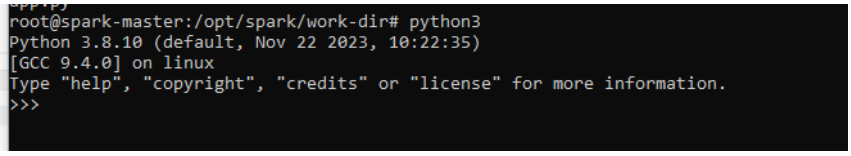


FIGURE 3.3 – Python 3.8.10 disponible dans le conteneur spark-master

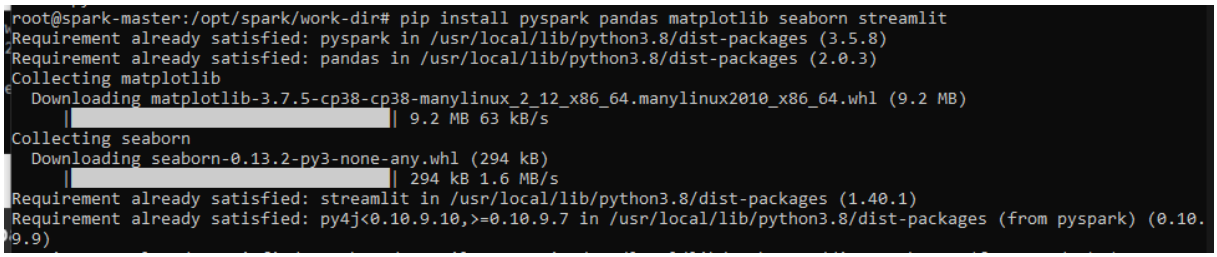


FIGURE 3.4 – Commande d'installation des bibliothèques Python pour Spark

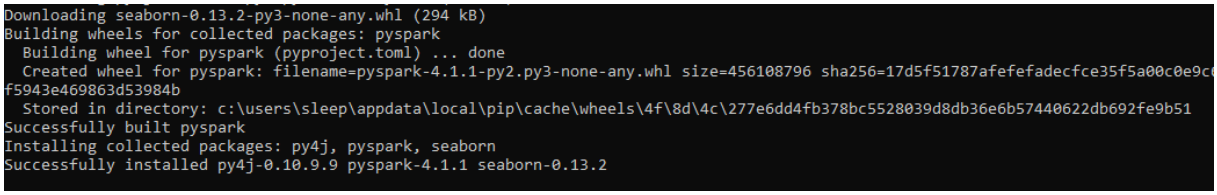


FIGURE 3.5 – Résultat de `pip install` — pyspark 4.1.1, seaborn 0.13.2 installés

3.2 Description du Dataset — Job Salary Prediction

3.2.1 Source Kaggle

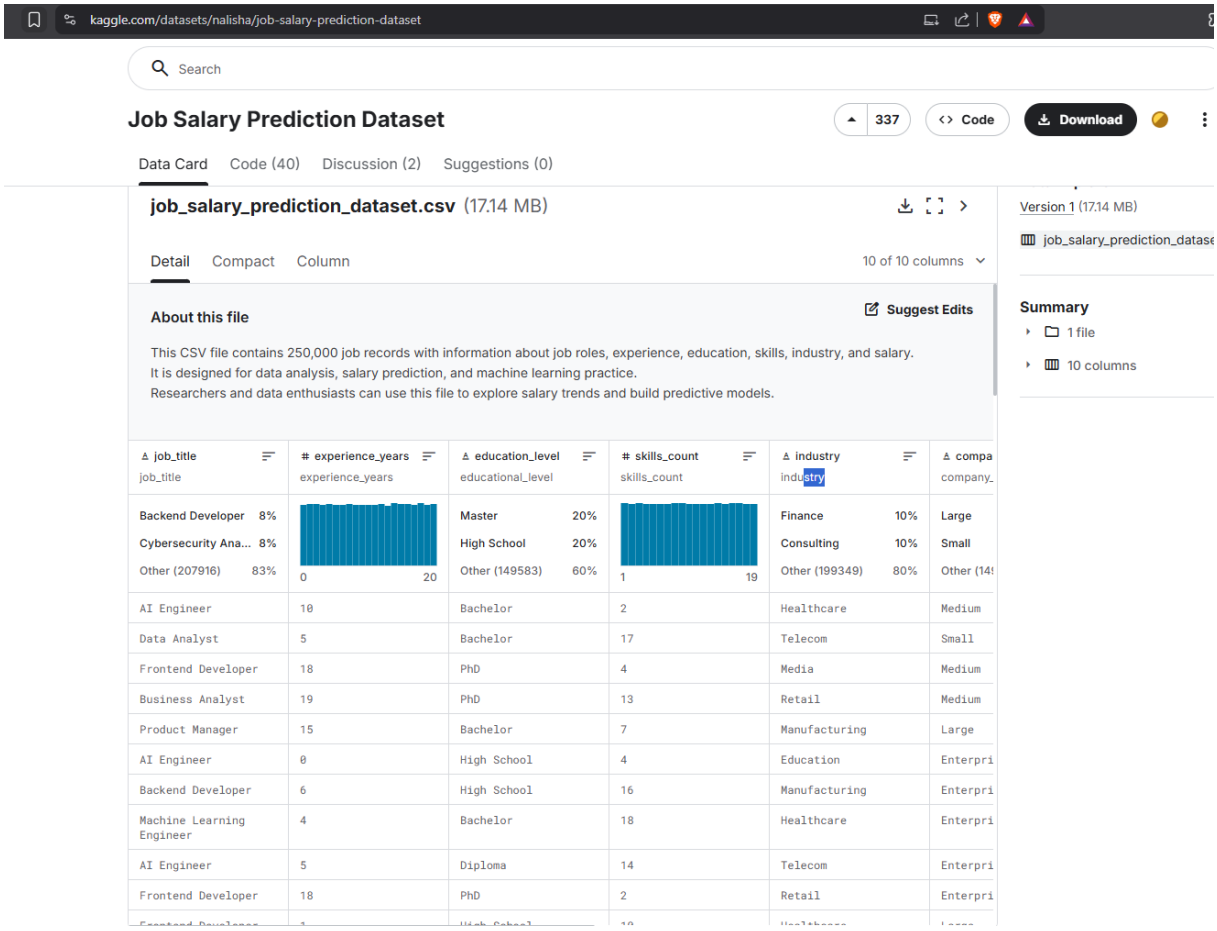


FIGURE 3.6 – Dataset *Job Salary Prediction* sur Kaggle — 250 000 enregistrements, 10 colonnes

TABLE 3.1 – Description des 10 colonnes du dataset

Colonne	Description
job_title	Intitulé du poste (AI Engineer, Data Analyst, Product Manager…)
experience_years	Années d'expérience (0 à 20)
education_level	Niveau d'études (High School, Diploma, Bachelor, Master, PhD)
skills_count	Nombre de compétences techniques maîtrisées (1 à 19)
industry	Secteur d'activité (Finance, Healthcare, Technology…)
company_size	Taille de l'entreprise (Small, Medium, Large, Enterprise)
location	Pays (USA, India, Canada, UK, Germany, Singapore…)
remote_work	Mode de travail (Yes = Remote, No = Présentiel, Hybrid)
certifications	Nombre de certifications professionnelles (0 à 5)
salary	Salaire annuel en dollars ($)

3.3 Script d'Analyse PySpark (spark.py)

3.3.1 Initialisation de la SparkSession

```
root@spark-master:/opt/spark/work-dir# /opt/spark/bin/spark-submit spark.py
26/05/10 09:53:44 INFO SparkContext: Running Spark version 3.5.1
26/05/10 09:53:44 INFO SparkContext: OS info Linux, 6.6.114.1-microsoft-standard-WSL2, amd64
26/05/10 09:53:44 INFO SparkContext: Java version 11.0.22
26/05/10 09:53:44 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes
 where applicable
26/05/10 09:53:45 INFO ResourceUtils: ==============================================================
26/05/10 09:53:45 INFO ResourceUtils: No custom resources configured for spark.driver.
26/05/10 09:53:45 INFO ResourceUtils: ==============================================================
26/05/10 09:53:45 INFO SparkContext: Submitted application: Salary Dashboard Analysis
26/05/10 09:53:46 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(cores -> name: cores, amount
: 1, script: , vendor: , memory -> name: memory, amount: 1024, script: , vendor: , offHeap -> name: offHeap, amount: 0, scrip
t: , vendor: ), task resources: Map(cpus -> name: cpus, amount: 1.0)
26/05/10 09:53:46 INFO ResourceProfile: Limiting resource is cpu
26/05/10 09:53:46 INFO ResourceProfileManager: Added ResourceProfile id: 0
26/05/10 09:53:46 INFO SecurityManager: Changing view acls to: root
26/05/10 09:53:46 INFO SecurityManager: Changing modify acls to: root
26/05/10 09:53:46 INFO SecurityManager: Changing view acls groups to:
26/05/10 09:53:46 INFO SecurityManager: Changing modify acls groups to:
26/05/10 09:53:46 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissio
ns: root; groups with view permissions: EMPTY; users with modify permissions: root; groups with modify permissions: EMPTY
26/05/10 09:53:46 INFO Utils: Successfully started service 'sparkDriver' on port 39281.
26/05/10 09:53:46 INFO SparkEnv: Registering MapOutputTracker
26/05/10 09:53:46 INFO SparkEnv: Registering BlockManagerMaster
26/05/10 09:53:46 INFO BlockManagerMasterEndpoint: Using org.apache.spark.storage.DefaultTopologyMapper for getting topology
information
26/05/10 09:53:46 INFO BlockManagerMasterEndpoint: BlockManagerMasterEndpoint up
26/05/10 09:53:46 INFO SparkEnv: Registering BlockManagerMasterHeartbeat
26/05/10 09:53:46 INFO DiskBlockManager: Created local directory at /tmp/blockmgr-ef508224-280c-46e4-ba74-753c8d327f74
26/05/10 09:53:46 INFO MemoryStore: MemoryStore started with capacity 434.4 MiB
```

FIGURE 3.7 – Connexion au spark-master et lecture du CSV depuis HDFS

```python
from pyspark.sql import SparkSession
import pyspark.sql.functions as F

# ============================================================
# SPARK SESSION
# ============================================================

spark = SparkSession.builder \
    .master("spark://spark-master:7077") \
    .appName("Salary Dashboard Analysis") \
    .getOrCreate()
```

Listing 3.2 – SparkSession

3.3.2 Lecture et inspection du CSV

```
from pyspark.sql import SparkSession
import pyspark.sql.functions as F

# ============================================================
# SPARK SESSION
# ============================================================

spark = SparkSession.builder \
    .master("spark://spark-master:7077") \
    .appName("Salary Dashboard Analysis") \
    .getOrCreate()
```

FIGURE 3.8 – Lecture du CSV depuis HDFS avec `inferSchema` et affichage du schéma

```
# ============================================================
# LOAD DATA
# ============================================================

df = spark.read.csv(
    "hdfs://namenode:9000/datasets/job_salary_prediction_dataset.csv",
    header=True,
    inferSchema=True
)
```

```
# ============================================================
# PRINT SCHEMA
# ============================================================

print("\n===== DATASET SCHEMA =====")
df.printSchema()

print("\n===== COLUMN NAMES =====")
print(df.columns)
```

FIGURE 3.9 – Code de lecture CSV et appels `df.printSchema()` / `df.show(10)`

3.3.3 Aperçu des données brutes

```
=== RAW SAMPLE (10 rows) ===
+----------------------+---------------+---------------+------------+-------------+------------+---------+-----------+--------------+------+
|job_title             |experience_years|education_level|skills_count|industry     |company_size|location |remote_work|certifications|salary|
+----------------------+---------------+---------------+------------+-------------+------------+---------+-----------+--------------+------+
|AI Engineer           |10             |Bachelor       |2           |Healthcare   |Medium      |India    |Hybrid     |2             |109413|
|Data Analyst          |5              |Bachelor       |17          |Telecom      |Small       |Australia|No         |0             |93764 |
|Frontend Developer    |18             |PhD            |4           |Media        |Medium      |Singapore|No         |1             |148123|
|Business Analyst      |19             |PhD            |13          |Retail       |Medium      |Canada   |Yes        |0             |189123|
|Product Manager       |15             |Bachelor       |7           |Manufacturing|Large       |Sweden   |Yes        |0             |165069|
|AI Engineer           |0              |High School    |4           |Education    |Enterprise  |USA      |No         |2             |180351|
|Backend Developer     |6              |High School    |16          |Manufacturing|Enterprise  |Canada   |No         |3             |165375|
|Machine Learning Engineer|4           |Bachelor       |18          |Healthcare   |Enterprise  |Canada   |Hybrid     |5             |202463|
|AI Engineer           |5              |Diploma        |14          |Telecom      |Enterprise  |Singapore|Yes        |0             |171834|
|Frontend Developer    |18             |PhD            |2           |Retail       |Enterprise  |India    |Hybrid     |5             |128377|
+----------------------+---------------+---------------+------------+-------------+------------+---------+-----------+--------------+------+
only showing top 10 rows
Rows: 250000
Columns: 10
Column names: ['job_title', 'experience_years', 'education_level', 'skills_count', 'industry', 'company_size', 'location', 'remote_work', 'certi
fications', 'salary']
```

FIGURE 3.10 – Échantillon des 10 premières lignes — 250 000 lignes, 10 colonnes confirmées

```
df = df_raw.dropDuplicates()
df = (
    df.withColumn("salary",           _safe_double("salary"))
      .withColumn("experience_years", _safe_int("experience_years")
         )
      .withColumn("skills_count",     _safe_int("skills_count"))
      .withColumn("certifications",   _safe_int("certifications"))
)
df = df.filter(F.col("salary")           > 0)
df = df.filter(F.col("experience_years") >= 0)
df = df.filter(F.col("skills_count")     >= 0)
df = df.filter(F.col("certifications")   >= 0)
```

Listing 3.3 – Nettoyage et filtrage des données

3.4 Analyse Descriptive — KPIs Globaux

```
st.subheader("KPI's")
kpis = df.agg(
    F.count("*").alias("records"),
    F.avg("salary").alias("avg_salary"),
    F.avg("experience_years").alias("avg_exp"),
    F.min("salary").alias("min_salary"),
    F.max("salary").alias("max_salary")
).collect()[0]
```

```
26/05/10 09:54:09 INFO CodeGenerator: Code generated in 8.234966 ms
+-------------+--------------+------------------+--------------+--------------+
|Total_Records|Average_Salary|Average_Experience|Minimum_Salary|Maximum_Salary|
+-------------+--------------+------------------+--------------+--------------+
|       250000| 145718.080524|         10.005408|       31867.0|      333046.0|
+-------------+--------------+------------------+--------------+--------------+
```

FIGURE 3.11 – Code PySpark pour le calcul des KPIs globaux

TABLE 3.2 – KPIs globaux obtenus sur les 250 000 enregistrements

KPI	Indicateur	Valeur
1	Nombre total d'employés	**250 000**
2	Salaire moyen	**$145 718.08**
3	Salaire minimum	**$31 867.00**
4	Salaire maximum	**$333 046.00**
5	Expérience moyenne	**10 ans**

Interprétation : Avec un salaire moyen de $145 718 et un écart important entre le minimum ($31 867) et le maximum ($333 046), la distribution révèle de fortes disparités selon le rôle, la localisation et le niveau d'expérience.

3.5 Résultats d'Analyse Détaillés

3.5.1 Top 10 des postes par salaire moyen

```
df.groupBy("job_title") \
  .agg(F.avg("salary").alias("avg_salary")) \
  .orderBy(F.col("avg_salary").desc()) \
  .limit(10).toPandas()
```

Listing 3.4 – Requête — Top 10 postes

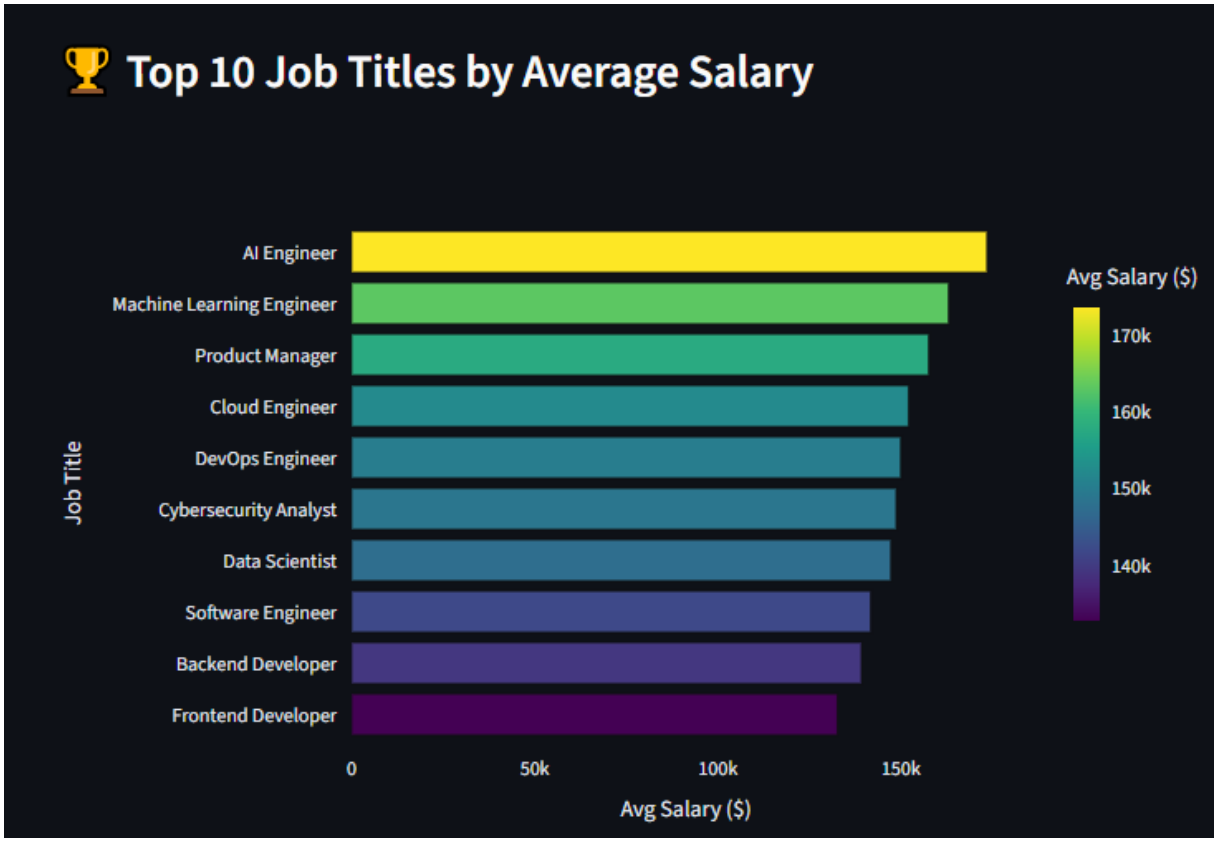


FIGURE 3.12 – Top 10 des postes par salaire moyen — AI Engineer en tête avec ≈$175k

TABLE 3.3 – Top 10 des postes par salaire moyen

Rang	Poste	Salaire moyen ($)
1	AI Engineer	≈175 000
2	Machine Learning Engineer	≈170 000
3	Product Manager	≈165 000
4	Cloud Engineer	≈163 000
5	DevOps Engineer	≈162 000
6	Cybersecurity Analyst	≈160 000
7	Data Scientist	≈155 000
8	Software Engineer	≈152 000
9	Backend Developer	≈148 000
10	Frontend Developer	≈145 000

Interprétation : Le poste d'*AI Engineer* est le mieux rémunéré, suivi du *Machine Learning Engineer* et du *Product Manager*. Les métiers liés à l'IA, au cloud et à la cybersécurité dominent le classement. En bas du top 10, on retrouve *Frontend Developer* et *Backend Developer*.

3.5.2 Top 10 des localisations par salaire moyen

```
df.groupBy("location") \
  .agg(F.avg("salary").alias("avg_salary")) \
  .orderBy(F.col("avg_salary").desc()) \
  .limit(10).toPandas()
```

Listing 3.5 – Requête — Top 10 localisations

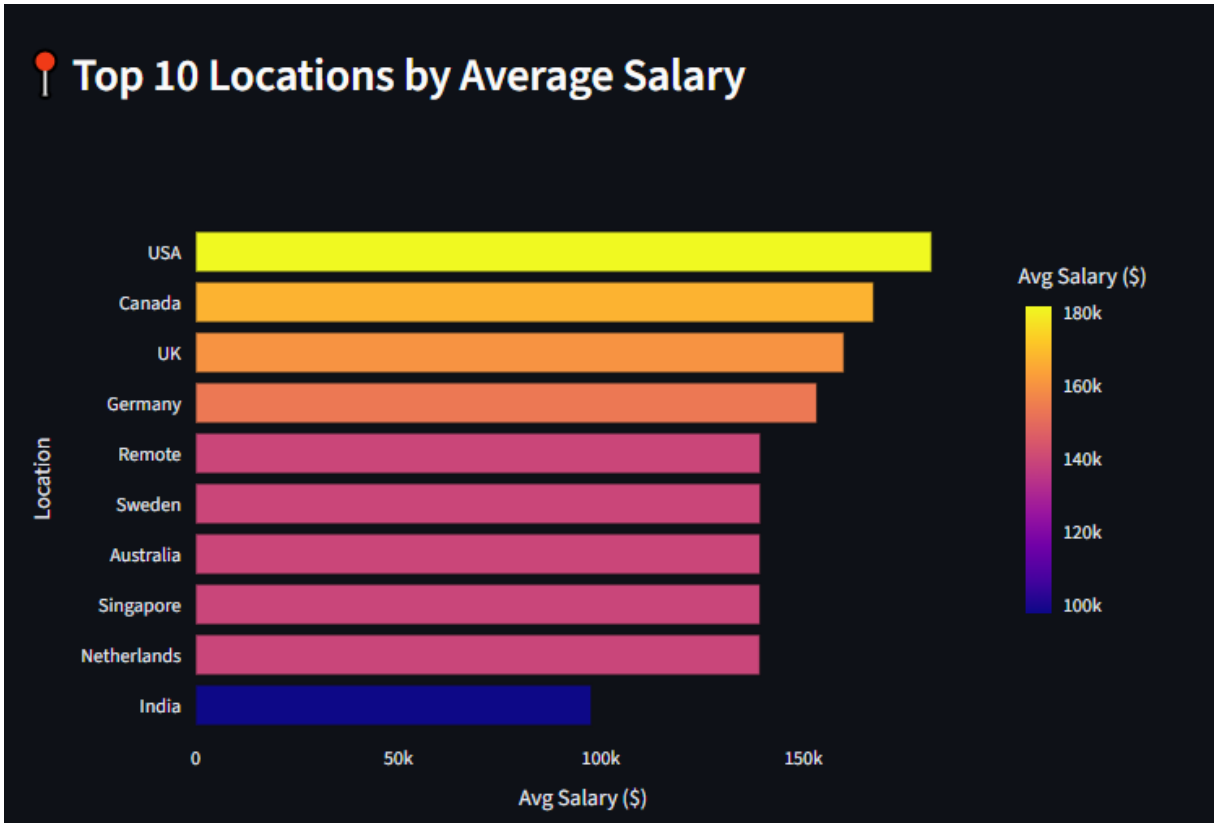


FIGURE 3.13 – Top 10 des localisations par salaire moyen — USA en tête, suivis du Canada et du UK

TABLE 3.4 – Top 10 des localisations par salaire moyen

Rang	Localisation	Salaire moyen ($)
1	USA	≈185 000
2	Canada	≈175 000
3	UK	≈168 000
4	Germany	≈165 000
5	Remote	≈162 000
6	Sweden	≈158 000
7	Australia	≈155 000
8	Singapore	≈153 000
9	Netherlands	≈150 000
10	India	≈105 000

Interprétation : Les États-Unis occupent la première place. Le mode *Remote* figure parmi les mieux classés (rang 5), montrant que le travail à distance peut offrir des salaires attractifs comparables aux grandes économies européennes.

3.5.3 Salaire moyen selon le statut de télétravail

```
df.groupBy("remote_work") \
  .agg(F.avg("salary").alias("avg_salary")) \
```

```
  .toPandas()
```

Listing 3.6 – Requête — Télétravail

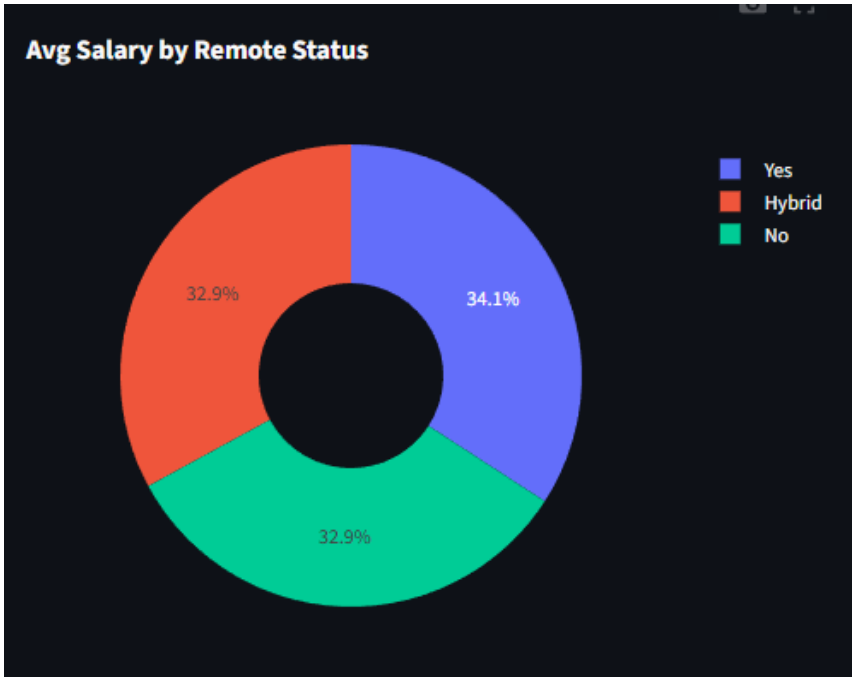


FIGURE 3.14 – Salaire moyen par statut de télétravail — Remote (34.1%), Hybrid (32.9%), Présentiel (32.9%)

TABLE 3.5 – Salaire moyen par mode de travail

Mode	Salaire Moyen ($)	Part (%)
Remote (Yes)	149 279	34.1%
Hybrid	143 969	32.9%
Présentiel (No)	143 932	32.9%

Interprétation : Le télétravail complet (Yes) représente 34.1% et bénéficie du salaire moyen le plus élevé (+$5 347 par rapport au présentiel). Les modes hybride et sur site sont quasi identiques (32.9% chacun).

3.5.4 Salaire moyen selon le niveau d'éducation

```
df.groupBy("education_level") \
  .agg(F.avg("salary").alias("avg_salary"),
       F.count("*").alias("records")) \
  .orderBy(F.col("avg_salary").desc()) \
  .toPandas()
```

Listing 3.7 – Requête — Niveau d'éducation

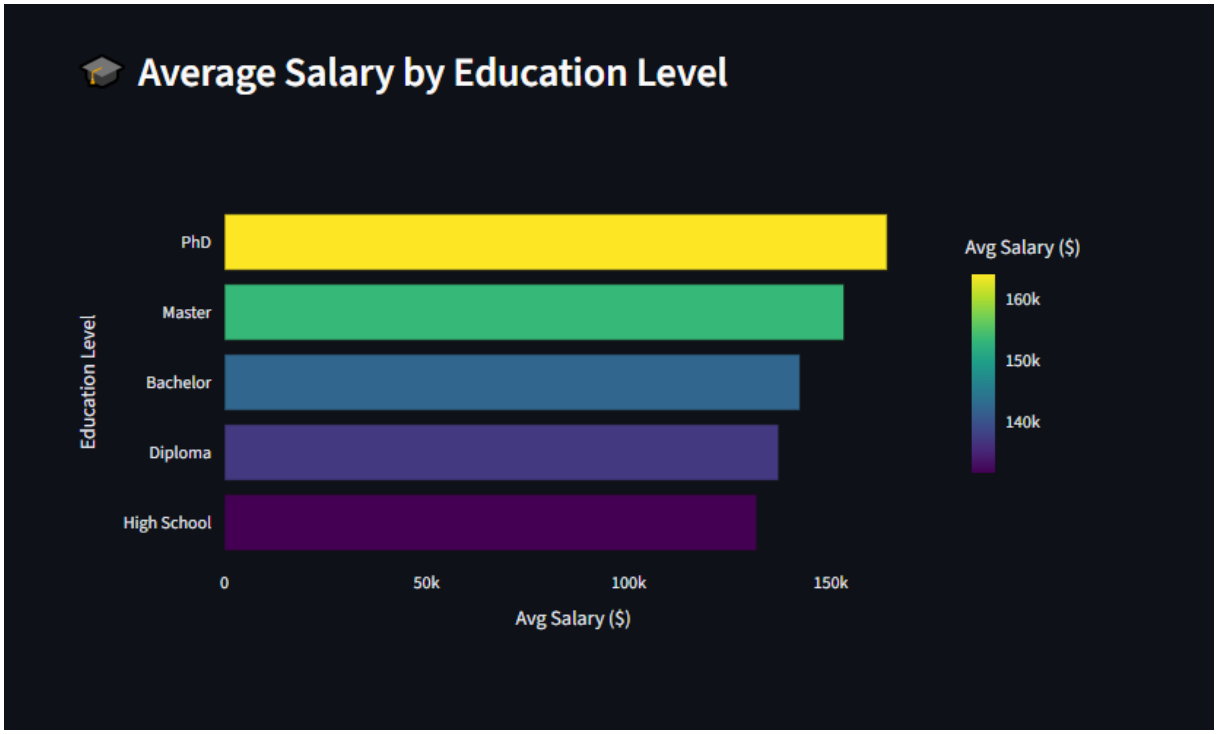


FIGURE 3.15 – Salaire moyen par niveau d'éducation — PhD en tête, High School le plus bas

Interprétation : Les titulaires d'un **PhD** bénéficient des salaires les plus élevés (≈$162k), suivis par les **Master** (≈$155k). Le niveau **High School** enregistre les rémunérations les plus faibles (≈$138k). L'éducation a un impact clair et progressif sur le niveau de salaire.

3.5.5 Top 10 des secteurs d'activité par salaire moyen

```
df.groupBy("industry") \
  .agg(F.avg("salary").alias("avg_salary"),
       F.count("*").alias("records")) \
  .orderBy(F.col("avg_salary").desc()) \
  .limit(10).toPandas()
```

Listing 3.8 – Requête — Secteurs d'activité

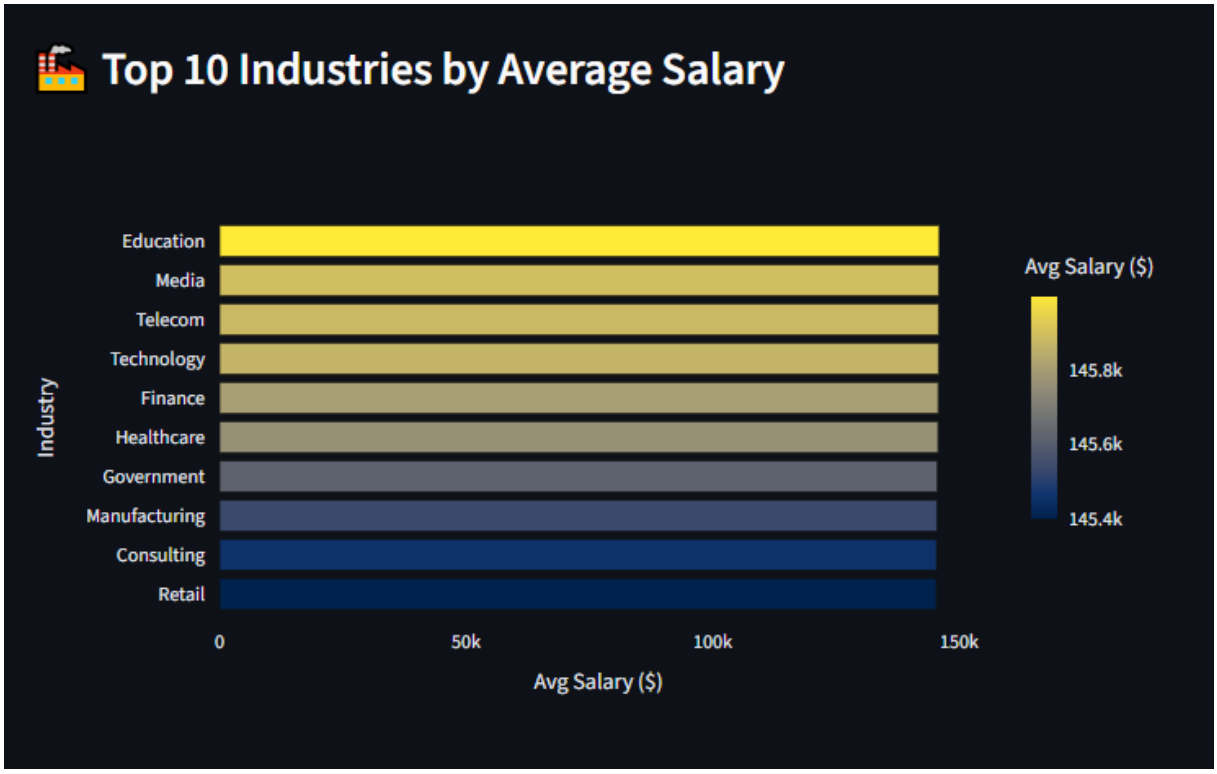


FIGURE 3.16 – Top 10 des secteurs par salaire moyen — écarts faibles, Education légèrement en tête

Interprétation : Les écarts de rémunération entre les industries sont très faibles (moins de $500 de différence). Le secteur **Éducation** apparaît légèrement en tête, suivi par **Médias**, **Télécoms** et **Technologie**. À l'inverse, **Retail** et **Consulting** affichent les salaires les plus bas.

3.5.6 Salaire moyen par nombre de certifications

```
df.groupBy("certifications") \
  .agg(F.avg("salary").alias("avg_salary"),
       F.count("*").alias("records")) \
  .orderBy(F.col("certifications").asc()) \
  .toPandas()
```

Listing 3.9 – Requête — Certifications

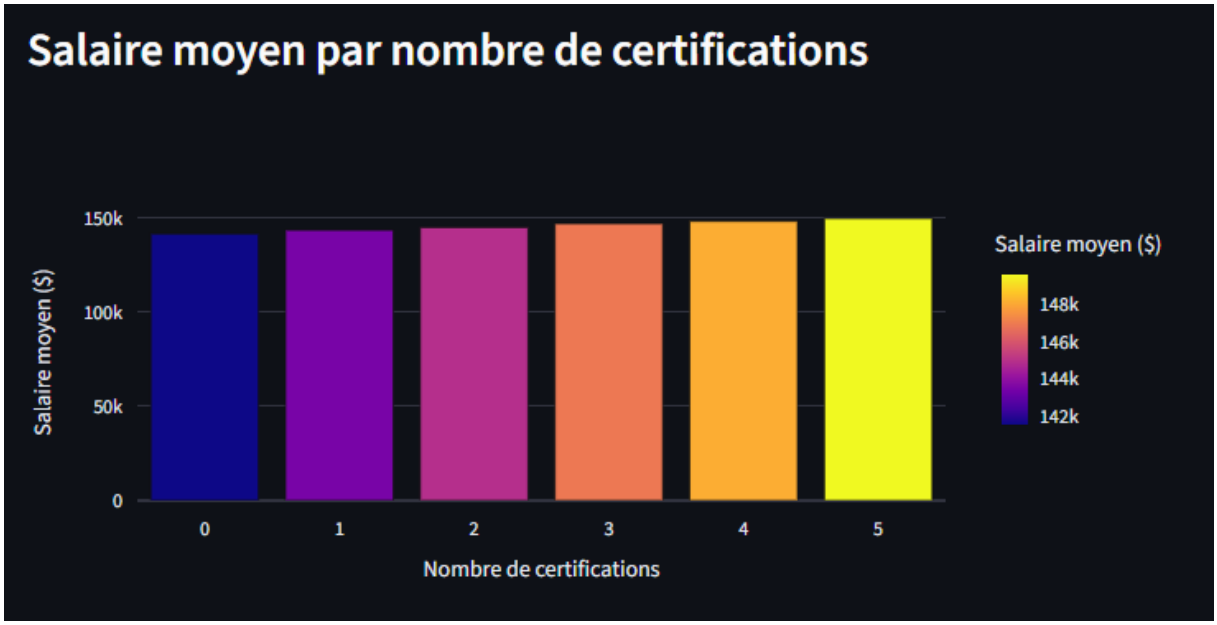


FIGURE 3.17 – Salaire moyen par nombre de certifications — progression de 0 à 5 certifications

Interprétation : On observe une progression régulière : de ≈$142k (0 certification) à ≈$149k (5 certifications). Chaque certification supplémentaire contribue positivement au salaire moyen, confirmant l'impact des certifications professionnelles sur la rémunération.

3.5.7 Tableau de bord Streamlit

```
python -m streamlit run app.py
```

Listing 3.10 – Lancement du dashboard Streamlit

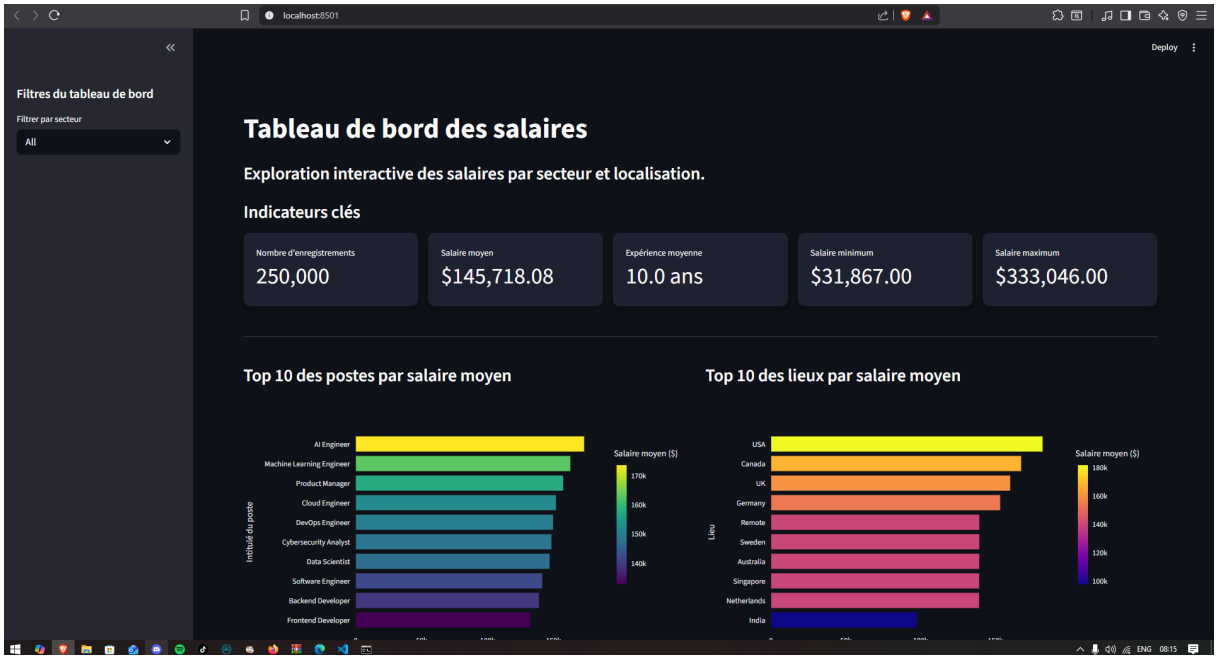


FIGURE 3.18 – Tableau de bord interactif Streamlit — KPIs, Top 10 postes et localisations

Le dashboard présente tous les KPIs et graphiques de façon interactive, avec un filtre par secteur d'activité dans la barre latérale.

Analyse Prescriptive

Sur la base des résultats, voici les recommandations formulées :

4.1 Pour les Entreprises

— Pour attirer des talents en IA : proposer le télétravail complet, recruter aux USA / Canada, financer des certifications professionnelles.

— Pour réduire le turnover des développeurs Frontend/Backend : revoir les packages salariaux et proposer des bonus de performance.

— Pour optimiser les coûts RH : recruter des profils juniors dans des pays à coût inférieur (Inde, Singapour) tout en maintenant la compétitivité.

4.2 Pour les Étudiants

— **Orientation :** viser les métiers IA/ML (les mieux rémunérés)

— **Formation :** accumuler des certifications professionnelles

— **Géographie :** privilégier USA, Canada ou mode Remote

— **Compétences :** multiplier les skills techniques

— **Études :** viser Master ou PhD pour maximiser le salaire

4.3 Pour les Équipes RH

— Intégrer les certifications dans les critères de promotion et d'augmentation

— Inclure le mode Remote ou Hybrid dans les offres pour attirer de meilleurs profils

— Revoir les packages Frontend si inférieurs au marché (bonus, stock-options)

Conclusion

5.1 Bilan du projet

Ce projet a mis en œuvre une chaîne complète Big Data, de l'infrastructure à la visualisation :

1. **Cluster Hadoop** déployé via Docker Compose (1 namenode + 3 datanodes + YARN)
2. **MapReduce** : comptage des transactions par ville sur `purchases.txt` (15.7 MB)
3. **PySpark** : analyse de 250 000 enregistrements salariaux avec nettoyage, agrégations et 6 analyses thématiques
4. **Streamlit** : dashboard interactif avec filtres et visualisations dynamiques

5.2 Principaux enseignements

TABLE 5.1 – Synthèse des résultats clés

Dimension	Résultat
Meilleur métier	AI Engineer (≈$175k)
Meilleur pays	USA (≈$185k)
Mode de travail	Remote légèrement supérieur (+$5k vs présentiel)
Éducation	PhD > Master > Bachelor (impact démontré)
Certifications	+$7k de 0 à 5 certifications
Secteur	Écarts très faibles (<$500) entre industries

Conclusion : Ce projet démontre la complémentarité d'Hadoop et Spark. Hadoop assure un stockage distribué fiable (HDFS) et le traitement par lots (MapReduce), tandis que Spark offre une analyse interactive beaucoup plus rapide grâce au traitement en mémoire. Ensemble, ils constituent le fondement de toute infrastructure Big Data à grande échelle.

Références et Bibliographie

1. Apache Hadoop Documentation. *HDFS Architecture*. https://hadoop.apache.org/docs/current/
2. Apache Spark Documentation. *PySpark API Reference*. https://spark.apache.org/docs/latest/api/python/
3. Docker Documentation. *Docker Compose Reference*. https://docs.docker.com/compose/
4. Big Data Europe. *bde2020 Docker images for Hadoop*. https://github.com/big-data-europe/docker-hadoop
5. Kaggle. *Job Salary Prediction Dataset*. https://www.kaggle.com/datasets/nalisha/job-salary-prediction-dataset
6. Streamlit Documentation. https://docs.streamlit.io/
7. White, T. (2015). *Hadoop : The Definitive Guide*. O'Reilly Media.
8. Zaharia, M. et al. (2016). Apache Spark : A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11), 56–65.